Capitolo 5

Refactoring e miglioramento della struttura del codice

5.1 Obiettivo e limite del refactoring

Il *refactoring* è il processo di trasformazione di un sistema software che ne migliora la struttura interna senza modificarne il comportamento esterno: dopo la trasformazione restano soddisfatti gli stessi requisiti funzionali. Per *struttura interna* si intendono, per esempio, la ripartizione delle responsabilità fra classi e metodi, la duplicazione e le dipendenze; per *comportamento esterno* ciò che osserva chi invoca il sistema, quali risultati, effetti e interfacce pubbliche previste dai requisiti.

Non è dunque refactoring né la correzione di un difetto né l'aggiunta di una funzionalità: entrambe cambiano il comportamento desiderato. Il refactoring è manutenzione del codice esistente e si applica a codice che già funziona. Una modifica che altera il comportamento per adattare il programma a nuove librerie o piattaforme è invece una diversa attività di manutenzione, anche se può essere accompagnata da refactoring.

La progettazione non è necessariamente adeguata al primo tentativo e tende a deteriorarsi quando si aggiungono modifiche. Il refactoring la migliora gradualmente durante e dopo l'implementazione, così che requisiti futuri siano più facili da incorporare. Può ridurre la quantità di codice, eliminare duplicazioni, rendere più leggibili strutture logiche con molti condizionali annidati, separare classi con troppe responsabilità e diminuire l'accoppiamento, cioè la dipendenza fra componenti.

Un beneficio del refactoring è preservare e migliorare la *manutenibilità*, cioè la facilità di comprendere, correggere e modificare il software. In particolare, il principio DRY (*Don't Repeat Yourself*) richiede di non duplicare codice: se un frammento copiato contiene un difetto, la correzione va ripetuta in ogni copia e le copie possono divergere. Una buona struttura rende anche più semplice individuare difetti, non perché il refactoring li corregga per definizione, ma perché richiede di comprendere più profondamente il codice.

Il *debito tecnico* è la conseguenza futura di una soluzione rapida ma poco curata: nel tempo si pagano "interessi" sotto forma di maggiore tempo per comprendere e modificare il codice. Una scadenza può indurre a scrivere una soluzione poco leggibile; non migliorarla quando possibile aumenta tale debito. Il refactoring è quindi un intervento preventivo contro il degrado della progettazione, non una garanzia che il sistema sia corretto o che il codice diventi sempre più breve.

Il refactoring va distinto dal *reverse engineering*, l'analisi di un sistema per ricavarne informazioni su comportamento o struttura, e dal *re-engineering*, la ricostruzione di un sistema con struttura differente. Queste attività possono essere utili su software esistente, ma hanno obiettivi distinti: il tratto essenziale del refactoring è la preservazione del comportamento.

Prima della trasformazione occorre avere test superati sul comportamento che deve rimanere invariato; dopo ciascun piccolo passo si compila ed esegue nuovamente la suite di test. Una *suite* è l'insieme dei casi di test eseguiti insieme. Se un test fallisce, il passo ha presumibilmente compromesso un comportamento già funzionante: si interviene subito, invece di accumulare molte modifiche e rendere difficile localizzare l'errore.

I test sono una rete di sicurezza, non una dimostrazione assoluta: proteggono soltanto i comportamenti che osservano e il cui oracle è corretto. L'*oracle* è il riferimento per decidere se un risultato è atteso; nel refactoring, l'output della versione precedente può svolgere questo ruolo. Se il codice iniziale non è corretto, il refactoring non è il modo per correggerlo: prima si chiarisce il comportamento richiesto e si corregge il difetto con test adeguati, poi si migliora la struttura.

Nel ciclo TDD il refactoring segue il test superato: Red individua un comportamento mancante, Green lo realizza e Refactor ne migliora la struttura mantenendo verdi i test. La revisione fra due programmatori che lavorano insieme, detta *pair programming*, può aiutare a discutere le trasformazioni e ad affrontarle con maggiore fiducia. Questa collaborazione, i test frequenti e il design semplice si sostengono reciprocamente.

5.1.1 Invarianza e sequenza di lavoro

La conservazione del comportamento può essere espressa come un'invariante: se P è il programma prima della trasformazione, P' quello dopo e x un input del dominio considerato, l'obiettivo è

$$\mathrm{obs}(P, x) = \mathrm{obs}(P', x). \tag{5.1}$$

La funzione $\mathrm{obs}(P, x)$ rappresenta il comportamento osservabile di P sull'input x, compresi risultato ed effetti previsti. L'uguaglianza deve valere per tutti gli input rilevanti; una suite finita la controlla soltanto per i casi che esegue.

Un procedimento operativo comune alle tecniche presentate è:

1. identificare un sintomo di debolezza del progetto e il comportamento che deve restare invariato;
2. verificare che il codice attuale funzioni e che esista una suite di regressione significativa;
3. scegliere una tecnica e compiere un solo piccolo cambiamento strutturale;
4. compilare per intercettare errori di tipo, visibilità e riferimenti;
5. eseguire tutti i test; se falliscono, correggere o annullare subito l'ultimo passo;
6. ripetere fino a ottenere una struttura più semplice, senza introdurre funzionalità nuove.

5.2 Code smell e scelta della trasformazione

Un *code smell*, o *bad smell*, è un sintomo nel codice che suggerisce una possibile debolezza della progettazione e invita a esaminarla; non è, da solo, la prova di un difetto. La scelta della trasformazione dipende dal sintomo:

codice duplicato lo stesso comportamento appare in più punti; si può usare *Estrai Metodo*;

metodo lungo o con commenti interni il commento spesso segnala un blocco con un intento nominabile; si usa *Estrai Metodo*, spesso preceduto da *Sostituisci Temp con Query*;

classe grande una classe con molti attributi, molte linee o più gruppi di responsabilità suggerisce *Estrai Classe*;

lista di parametri lunga si può usare *Introduci Oggetto Parametro*, cioè raggruppare dati correlati in un oggetto parametro;

cambiamenti divergenti parti della stessa classe cambiano per ragioni diverse; si usa *Estrai Classe* per separare le responsabilità.

Una *responsabilità* è un compito o motivo di cambiamento coerente affidato a un elemento del programma. La scelta è guidata dall'intento e dalle dipendenze effettive, non da una soglia numerica universale di righe, parametri o attributi.

5.3 Estrai Metodo

Estrai Metodo raggruppa un frammento in un nuovo metodo il cui nome ne comunica l'intenzione. È adatto a metodi lunghi, codice duplicato o blocchi che richiedono commenti per essere capiti. I metodi piccoli e ben nominati sono più riusabili; il metodo di alto livello diventa una sequenza leggibile di chiamate e l'override, cioè la ridefinizione di un metodo in una sottoclasse, risulta più agevole.

Il procedimento è il seguente:

1. individuare il frammento e scegliere un nome che descriva ciò che fa, non soltanto come lo fa;
2. copiare il frammento nel nuovo metodo;
3. trasformare in parametri le variabili locali del metodo sorgente che il frammento legge;
4. dichiarare nel nuovo metodo le variabili usate soltanto dal frammento;
5. se il frammento modifica una variabile locale del metodo sorgente, valutare se il nuovo metodo possa restituire il valore aggiornato come query;
6. sostituire il frammento nel metodo sorgente con la chiamata al nuovo metodo, quindi compilare e testare.

Una *query* restituisce un'informazione senza modificare lo stato osservabile dell'oggetto. Il passaggio 5 non è automatico: se non è possibile esprimere correttamente il risultato mediante un valore restituito, bisogna riesaminare il confine dell'estrazione.

La classificazione delle variabili precede lo spostamento del frammento. Se gli output sono molti, prima conviene separare le responsabilità: aggiungere campi alla classe soltanto per aggirare lo scope locale introduce stato non necessario. La lunghezza è un indizio: tagliare righe senza ottenere un'operazione coerente non è sufficiente.

5.3.1 Esempio: separare stampa, calcolo e dettagli

Si consideri `printDovuto`, che stampa un banner, calcola il totale degli ordini e stampa i dettagli. La classe Ordine di questo esempio espone l'importo come query; il costruttore permette di preparare una fixture.

```
import java.util.List;

class Ordine {
    private final double importo;
    Ordine(double importo) { this.importo = importo; }
    public double getImporto() { return importo; }
}
class Stampe {
    private final List<Ordine> ordini;
```

```
    private final String nome = "Mike";
    Stampe(List<Ordine> ordini) { this.ordini = ordini; }
    public void printDovuto() {
        System.out.println("----------------");
        System.out.println("- Cliente Dare -");
        System.out.println("----------------");
        double tot = 0;
        for (Ordine o : ordini) tot += o.getImporto();
        System.out.println("nome: " + nome);
        System.out.println("tot: " + tot);
    }
}
```

Dopo l'estrazione, la stessa sequenza si legge nei nomi dei metodi:

```
class Stampe2 {
    private final List<Ordine> ordini;
    private final String nome = "Mike";
    Stampe2(List<Ordine> ordini) { this.ordini = ordini; }
    public void printDovuto() {
        printBanner();
        double tot = getTotale();
        printDettagli(tot);
    }
    public double getTotale() {
        double tot = 0;
        for (Ordine o : ordini) tot += o.getImporto();
        return tot;
    }
    public void printBanner() {
        System.out.println("----------------");
        System.out.println("- Cliente Dare -");
        System.out.println("----------------");
    }
    public void printDettagli(double somma) {
        System.out.println("nome: " + nome);
        System.out.println("tot: " + somma);
    }
}
```

`tot` resta locale al calcolo e `somma` diventa parametro della stampa. Nessun campo temporaneo è stato aggiunto per far comunicare i metodi. Con gli stessi ordini si confronta l'intero testo prodotto dalle due classi: banner, nome e totale devono restare uguali.

5.3.2 Applicare l'estrazione a Pagamenti

Nel metodo monolitico di lettura degli importi si riconoscono blocchi con confini diversi: apertura e lettura del file, controllo dei duplicati, conversione, somma e massimo. Estrarre `inserisci` localizza la politica dei duplicati; estrarre `leggiFile` separa l'I/O; `converti` isola la trasformazione del testo. I cicli di somma e massimo diventano operazioni autonome, mentre i getter osservano i risultati. L'ordine delle dipendenze chiarisce quali dati devono già esistere a ciascun passo.

Il passaggio successivo alle query elimina `totale` e `massimo`: il client non deve più ricordare di ricalcolarli dopo ogni inserimento. Per dimostrare che una trasformazione è solo strutturale si confrontano risultati ed effetti sugli stessi input e sulla stessa sequenza ammessa dal contratto precedente. Il passaggio da deduplicazione testuale a numerica, la modifica della propagazione

delle eccezioni o la correzione del massimo per dati negativi cambiano invece il contratto o il comportamento in alcuni casi: vanno trattati come decisioni separate, con test specifici. La sola somiglianza dei risultati su tre importi non prova equivalenza per tutte le stringhe e tutti gli errori.

5.4 Sostituisci Temp con Query

Sostituisci Temp con Query si applica quando una variabile temporanea conserva il risultato di un'espressione. Una *variabile temporanea* è una variabile locale al metodo nel quale è dichiarata; perciò altri metodi non possono leggerla direttamente. Si estrae l'espressione in una query e si sostituiscono gli usi della variabile con chiamate alla query. Il dato diventa così disponibile anche ad altri metodi e ciò facilita una successiva estrazione di metodi.

La tecnica è appropriata quando la temporanea viene assegnata una sola volta. Si dichiara dapprima `final` per far controllare al compilatore che non vi siano riassegnazioni, si compila, si estrae la parte destra dell'assegnazione in un metodo e si sostituiscono tutte le occorrenze. Se la variabile cambia valore, una query non rappresenta più un valore fisso: si può dividere la variabile in valori distinti oppure non applicare questa tecnica. In particolare, una variabile di accumulo, quale un contatore o una somma in un ciclo, non va trasformata meccanicamente in query. Un eventuale costo di ricalcolo va ottimizzato solo se diventa un problema reale osservato.

La sostituzione richiede anche che rivalutare l'espressione conservi il significato: se i dati da cui dipende cambiano tra due utilizzi, una query può dare un risultato diverso dalla fotografia conservata nella temporanea. Va quindi controllata l'assenza di effetti indesiderati della rivalutazione.

L'esempio seguente sostituisce le temporanee `prezzoBase` e `sconto` con query. `quantita` è il numero di unità e `prezzo` il prezzo unitario; il prodotto `quantita * prezzo` è il prezzo base. Poiché entrambe le espressioni dipendono dallo stato dell'oggetto e non da una variabile locale che cambia, possono essere richiamate dal metodo pubblico.

```
public double getPrezzo() {
    return prezzoBase() * sconto();
}

private double prezzoBase() {
    return quantita * prezzo;
}

private double sconto() {
    if (prezzoBase() > 1000) return 0.95;
    return 0.98;
}
```

5.5 Dividi Variabile Temp

Dividi Variabile Temp riguarda invece una temporanea riassegnata per rappresentare valori concettualmente diversi, ma non una variabile aggiornata in un ciclo né un accumulatore. Si dà un nome e una variabile separata a ciascun significato: ciascuna responsabilità diventa leggibile e può essere dichiarata `final`. Si rinomina la prima dichiarazione e i suoi riferimenti fino alla successiva assegnazione, si introduce allora la nuova variabile, si compila e testa, e si ripete per passi piccoli.

Un diverso esempio mostra perché dividere una temporanea: la stessa variabile non deve prima indicare il perimetro e poi l'area. Con larghezza `width` e altezza `height`, i valori distinti

sono

$$p = 2(\texttt{width} + \texttt{height}), \qquad a = \texttt{width} \cdot \texttt{height}, \tag{5.2}$$

dove p è il perimetro e a l'area. Nel codice i nomi `perim` e `area`, dichiarati `final`, rendono visibili i due significati. L'eccezione resta l'accumulatore di un ciclo: lì la riassegnazione rappresenta intenzionalmente un unico valore progressivo.

5.5.1 Due fasi del moto e due accelerazioni

Un corpo di massa `mass` è soggetto inizialmente a `primaryForce`; dopo `delay` si aggiunge `secondaryForce`. Il metodo calcola la distanza dopo `time`. Nel dominio fisico considerato massa positiva, tempi non negativi e ritardo intero permettono di leggere direttamente le formule; i cast a `int` del listato troncano invece eventuali frazioni di tempo e fanno parte del suo comportamento.

```
private double primaryForce, secondaryForce, mass, delay;

public double getDistanceTravelled1(int time) {
    double acc = primaryForce / mass;
    int primaryTime = (int) Math.min(time, delay);
    double result = 0.5 * acc * primaryTime * primaryTime;
    int secondT = (int) (time - delay);
    if (secondT > 0) {
        double primaryVel = acc * delay;
        acc = (primaryForce + secondaryForce) / mass;
        result += primaryVel * secondT
                + 0.5 * acc * secondT * secondT;
    }
    return result;
}
```

`acc` indica prima l'accelerazione iniziale e poi quella della seconda fase. La prima è $a_1 = F_1/m$; la seconda $a_2 = (F_1 + F_2)/m$. Per tempi interi coerenti con il ritardo, posto $t_1 = \min(t, d)$ e $t_2 = t - d > 0$, la distanza è $s = \frac{1}{2}a_1 t_1^2$ prima del cambio; dopo il cambio si aggiunge $v_1 t_2 + \frac{1}{2}a_2 t_2^2$, con $v_1 = a_1 d$. La velocità iniziale della seconda fase non deve essere persa.

Si divide la temporanea senza cambiare gli altri calcoli:

```
public double getDistanceTravelled2(int time) {
    final double primAcc = primaryForce / mass;
    int primaryTime = (int) Math.min(time, delay);
    double result = 0.5 * primAcc * primaryTime * primaryTime;
    int secondT = (int) (time - delay);
    if (secondT > 0) {
        double primaryVel = primAcc * delay;
        final double secondAcc = (primaryForce + secondaryForce) / mass;
        result += primaryVel * secondT
                + 0.5 * secondAcc * secondT * secondT;
    }
    return result;
}
```

`primAcc` e `secondAcc` hanno significati separati e sono finali. `result` resta un accumulatore: le sue assegnazioni costruiscono intenzionalmente la stessa distanza e non richiedono di dividerlo. Test prima del ritardo, al ritardo e dopo il ritardo verificano che la trasformazione conservi entrambi i rami.

5.6 Sposta Metodo

Sposta Metodo si applica quando un metodo usa più attributi e operazioni di un'altra classe che della classe nella quale risiede: il comportamento mostra una dipendenza eccessiva verso la classe destinataria. Si esaminano prima attributi e metodi usati, valutando se quelli impiegati soltanto dal comportamento trasferito debbano seguirlo; si controlla inoltre che il metodo non sia dichiarato in superclassi o sottoclassi. Si dichiara e copia il metodo nella classe destinataria, si adattano i riferimenti alle due classi e si mantiene provvisoriamente il vecchio metodo come *delegato*, cioè un metodo che inoltra la chiamata al nuovo. Dopo compilazione e test si decide se rimuovere il delegato o conservarlo.

Il polimorfismo richiede cautela. Se il metodo è dichiarato anche nella gerarchia di ereditarietà e il comportamento polimorfico appartiene concettualmente alla classe destinataria, lo spostamento dell'intero comportamento diventa un refactoring più complesso. Se invece quel comportamento deve rimanere nella classe originaria, il metodo non va spostato integralmente; si può estrarre soltanto parte della logica.

5.7 Estrai Classe

Estrai Classe separa una classe che svolge il lavoro di due classi. Il Principio di Singola Responsabilità (SRP) richiede che una classe abbia un solo motivo per cambiare. Sono indizi un sottogruppo di dati e metodi che stanno insieme, dati che cambiano insieme o dipendono fra loro, oppure una sottoclasse che usa solo poche caratteristiche della superclasse. Si decide la responsabilità da separare, si crea una nuova classe, si inserisce nella classe originaria un attributo che ne conserva un'istanza e si spostano attributi e metodi, iniziando da quelli di più basso livello. La nuova relazione è una dipendenza fra le due classi.

Una classe `Persona` può contenere prefisso e numero di telefono, dati che costituiscono un concetto separabile. *Estrai Classe* crea, per esempio, `Telefono` e lascia a `Persona` un riferimento a quell'oggetto:

```
class Telefono {
    private String prefisso;
    private String numero;

    String formatoCompleto() { return prefisso + " " + numero; }
}

class Persona {
    private Telefono telefono;
}
```

L'esempio non afferma che ogni attributo debba essere estratto: la separazione è giustificata perché i dati e le operazioni sul recapito sono una responsabilità coerente che può cambiare indipendentemente da quelle della persona.

5.8 Oggetto Parametro e sostituzione dei condizionali

Una lunga lista di parametri correlati può essere ridotta mediante *Introduci Oggetto Parametro*. Si raggruppano dati che formano un concetto coerente: invece di ripetere data iniziale e finale in più firme, si può introdurre un valore `Periodo`, concentrando nel suo costruttore la validazione dell'ordine delle date. Un contenitore di parametri non correlati nasconde la lunga firma senza chiarire la responsabilità. Durante il refactoring si possono scoprire bug, ma correggerli cambia il comportamento: conviene distinguerne il passo, aggiornare l'oracle e proteggere la correzione con un caso di regressione.

Quando molti metodi ripetono lo stesso `switch` per distinguere gli stati di un oggetto, si può trasferire ogni comportamento in una classe di stato e delegare attraverso un contratto comune. La selezione passa così dal confronto su un codice al polimorfismo. La trasformazione è detta *Replace Conditional with Polymorphism*; il caso specifico *Replace Type Code with State* porta alla struttura del pattern State.